Deep Learning Assignment 4

Deepfake Detection using ResNet-34 and ResNet-152

Anumalasetty Sohan Kumar CS23B1004
Shishir Kumar Reddy Ambala CS23B2043

Department of Computer Science and Engineering

March 8, 2026

Contents

1 Objective **2**

2 Dataset Description **2**

3 Part 1 — ResNet-34 (Manual Implementation) **2**
- 3.1 Architecture . . . 2
- 3.2 Parameter Verification . . . 2
- 3.3 Experiments . . . 3
  - 3.3.1 Experiment 1 — Adam, lr=1e-3, Batch=32 . . . 3
  - 3.3.2 Experiment 2 — SGD (momentum=0.9), lr=1e-3, Batch=32 . . . 3
  - 3.3.3 Experiment 3 — Adam, lr=1e-4, Batch=64 . . . 4
- 3.4 Test Set Evaluation — ResNet-34 . . . 4
- 3.5 Observations — ResNet-34 . . . 5

4 Part 2 — ResNet-152 (Pretrained, Progressive Fine-Tuning) **6**
- 4.1 Architecture and Fine-Tuning Strategy . . . 6
- 4.2 Experiments . . . 6
  - 4.2.1 Experiment 1 — Adam (lr: 1e-3 → 2e-4) . . . 6
  - 4.2.2 Experiment 2 — SGD with Momentum (lr: 1e-2 → 5e-4) . . . 6
- 4.3 Test Set Evaluation — ResNet-152 . . . 7
- 4.4 Observations — ResNet-152 . . . 8

5 Model Comparison **8**

6 Summary of Learnings **8**

7 Conclusion **9**

Objective

The objective of this assignment is to develop deep learning models capable of detecting deepfake images using convolutional neural networks. Two architectures are implemented and evaluated:

- **ResNet-34** — manually implemented using basic PyTorch layers (`Conv2d`, `BatchNorm2d`, `ReLU`, `MaxPool2d`, `Linear`)
- **ResNet-152** — pretrained on ImageNet, adapted via progressive fine-tuning

The goal is to classify images into two categories: **Real** and **Fake (Deepfake)**.

Dataset Description

The dataset used consists of images from two sources: **FaceForensics++ (FFPP)** and **CelebDF**, labelled as real or fake. A custom `DeepfakeDataset` class was implemented in PyTorch to load images and labels from the metadata CSV files.

- Images are resized to 224×224 pixels
- Training data uses augmentation: random horizontal flip, rotation ($\pm 10°$), and colour jitter
- Validation and test data use only resize and ImageNet normalization
- Class-weighted `CrossEntropyLoss` is used to handle class imbalance

Part 1 — ResNet-34 (Manual Implementation)

Architecture

The ResNet-34 architecture was implemented from scratch using basic PyTorch primitives — no pre-built `torchvision.models` were used. The model follows the standard ResNet-34 specification:

- **Initial Conv**: 7×7, stride 2, followed by BatchNorm, ReLU, and MaxPool
- **Layer 1**: 3 BasicBlocks, 64 channels
- **Layer 2**: 4 BasicBlocks, 128 channels, stride 2
- **Layer 3**: 6 BasicBlocks, 256 channels, stride 2
- **Layer 4**: 3 BasicBlocks, 512 channels, stride 2
- **Head**: Global Average Pooling $\rightarrow$ `Linear(512, 2)`

Each **BasicBlock** consists of two 3×3 convolutions with BatchNorm and ReLU, plus a skip connection. A **downsample** projection is applied when dimensions change.

Parameter Verification

The total parameter count was printed and verified to match the standard ResNet-34 specification of approximately **21.8 million parameters**.

Experiments

Three experiments were conducted by varying the optimizer, learning rate, and batch size. Each model was trained for **5 epochs**.

Table 1: ResNet-34 Experiment Configurations and Validation Results

Experiment	Optimizer	LR	Batch	Best Val Accuracy
Experiment 1	Adam	1e-3	32	0.8636
Experiment 2	SGD (mom=0.9)	1e-3	32	0.8651
Experiment 3	Adam	1e-4	64	**0.8717**

Experiment 1 — Adam, lr=1e-3, Batch=32

Training and validation loss decreased steadily. Validation accuracy reached **86.36%**. The higher learning rate allowed fast convergence but limited further improvement beyond epoch 3.

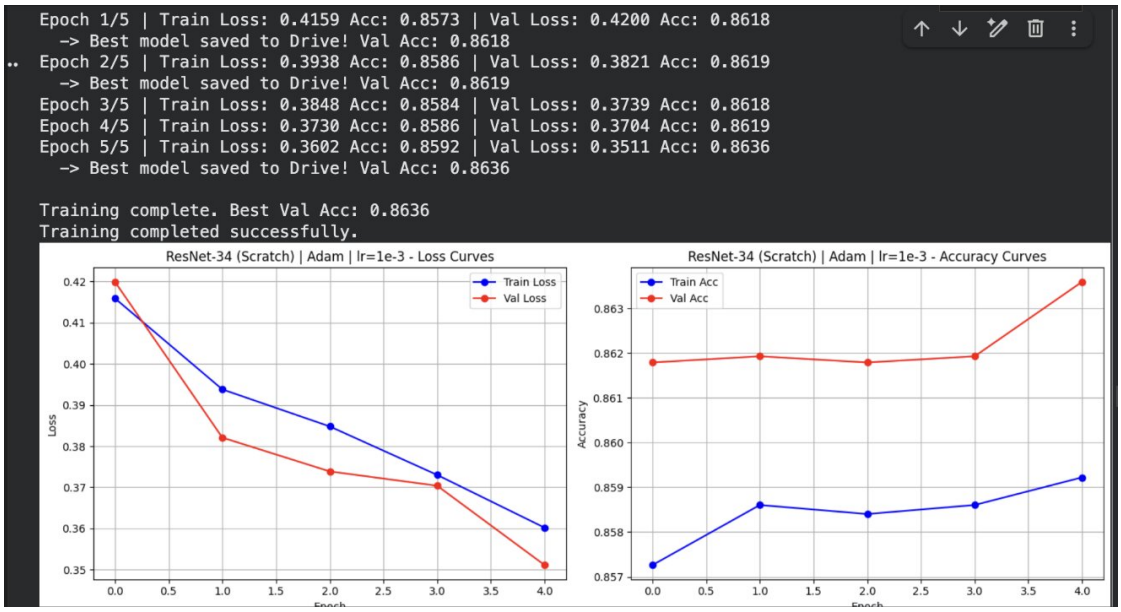


Figure 1: ResNet-34 — Adam — lr=1e-3 — Loss and Accuracy Curves

Experiment 2 — SGD (momentum=0.9), lr=1e-3, Batch=32

SGD converged slightly more slowly but reached a validation accuracy of **86.51%**, marginally higher than Experiment 1. The validation loss showed a dip at epoch 2 before recovering, indicating that SGD with higher LR introduces instability early on.

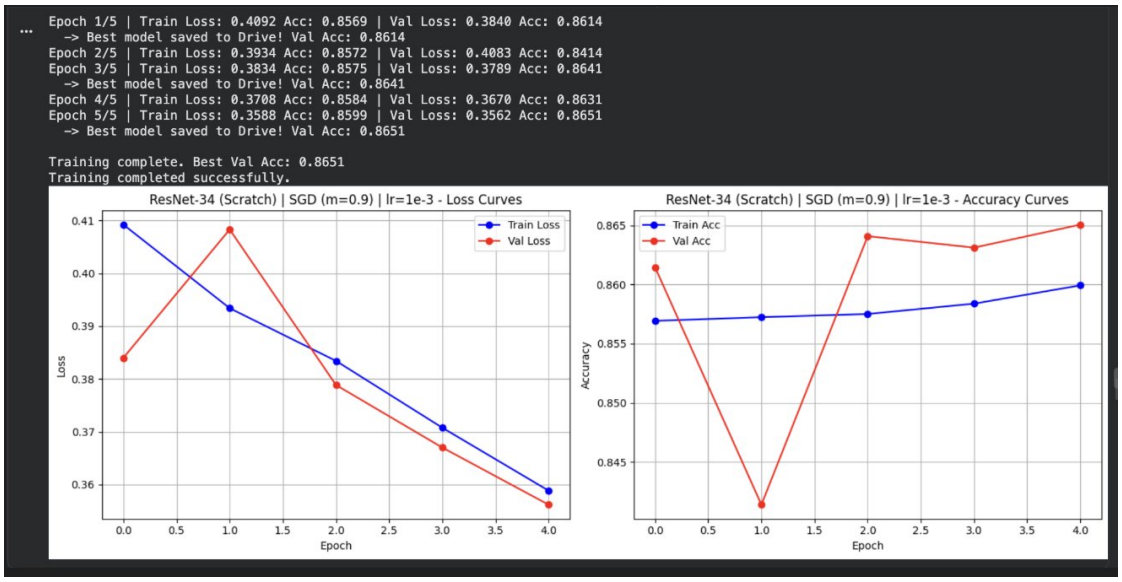


Figure 2: ResNet-34 — SGD (momentum=0.9) — lr=1e-3 — Loss and Accuracy Curves

Experiment 3 — Adam, lr=1e-4, Batch=64

Reducing the learning rate to `1e-4` significantly improved training stability. The larger batch size of 64 produced smoother gradient updates. This configuration achieved the highest validation accuracy of **87.17%** and was selected as the best model.

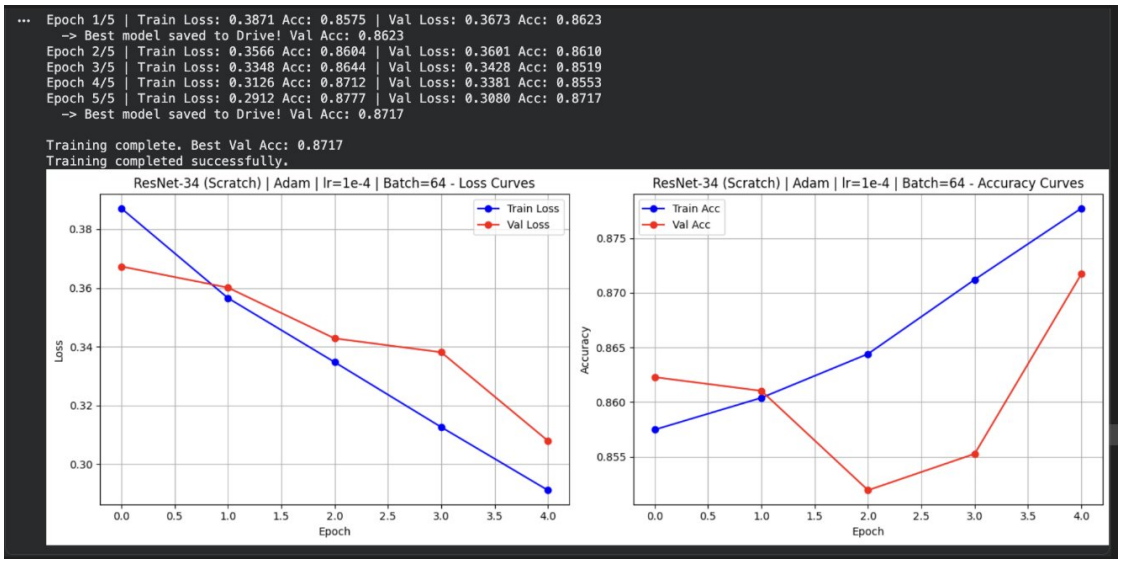


Figure 3: ResNet-34 — Adam — lr=1e-4, Batch=64 — Loss and Accuracy Curves

Test Set Evaluation — ResNet-34

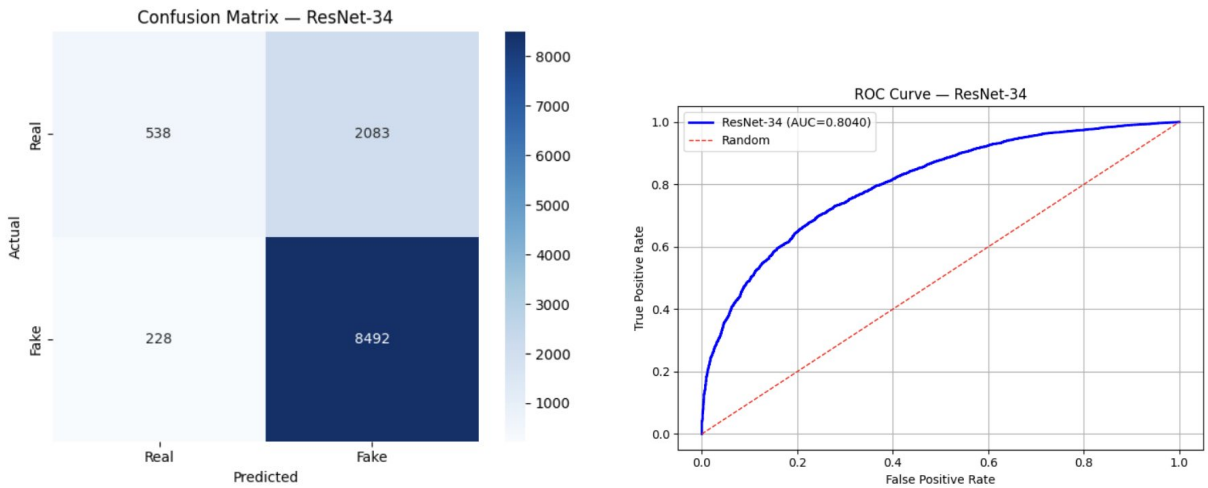
The best model (Experiment 3) was evaluated on the held-out test set. Since the dataset is imbalanced, **ROC-AUC is used as the primary metric**.

Table 2: ResNet-34 Test Set Metrics (primary metric: ROC-AUC)

Metric	Score	Note
ROC-AUC	**0.8040**	**Primary metric — imbalance-robust**
F1-Score	0.8802	Harmonic mean of precision & recall
Precision	0.8030	
Recall	0.9739	High — model is aggressive on Fake class
Accuracy	0.7962	Can be misleading under imbalance

Table 3: ResNet-34 Classification Report

Class	Precision	Recall	F1-Score	Support
Real	0.70	0.21	0.32	2621
Fake	0.80	0.97	0.88	8720
Overall Accuracy	**79.62%**			



(a) Confusion Matrix (b) ROC Curve (AUC = 0.8040)

Figure 4: ResNet-34 — Test Set Evaluation

Observations — ResNet-34

- The model achieves **very high recall for the Fake class (0.97)**, correctly identifying most deepfakes.
- Performance on the **Real class is lower** (recall = 0.21) due to class imbalance — fake samples dominate the dataset.
- A lower learning rate (`1e-4`) combined with a larger batch size (64) yielded the most stable and accurate training.
- SGD requires more careful LR tuning; at the same LR as Adam it shows oscillation in early epochs.

- Training from scratch with only 5 epochs limits the model's capacity — more epochs or a warm-up schedule would help.

Part 2 — ResNet-152 (Pretrained, Progressive Fine-Tuning)

Architecture and Fine-Tuning Strategy

A pretrained **ResNet-152** (ImageNet weights) was used. Instead of full fine-tuning — which is expensive — a **2-phase freezing strategy** was adopted to reduce training time significantly:

Table 4: 2-Phase Fine-Tuning Schedule (same structure for both experiments)

Phase	Layers Trained	Epochs	Trainable Params
Phase 1	FC head only (backbone fully frozen)	5	∼525K
Phase 2	FC head + layer4	5	∼13M

The custom classification head replaces the original FC layer with:

$$\begin{aligned} 2048 \;&\rightarrow\; \text{BatchNorm1d}(2048) \;\rightarrow\; \text{Dropout}(0.4) \;\rightarrow\; \text{Linear}(2048, 256) \\ &\rightarrow\; \text{ReLU} \;\rightarrow\; \text{Dropout}(0.2) \;\rightarrow\; \text{Linear}(256, 2) \end{aligned}$$

In Phase 1, the entire backbone is frozen and only the lightweight head (∼0.9% of total parameters) is trained, making each epoch very fast. In Phase 2, `layer4` is unfrozen, allowing the model to adapt high-level residual features to deepfake-specific artifacts. Total training is **10 epochs per experiment**.

Experiments

Table 5: ResNet-152 Experiment Configurations

Experiment	Optimizer	Phase 1 LR (FC only)	Phase 2 LR (FC + layer4)	Best Val AUC-ROC
Experiment 1	Adam	`1e-3`	`2e-4`	**0.9507**
Experiment 2	SGD (mom=0.9)	`1e-2`	`5e-4`	∼0.79

Experiment 1 — Adam (lr: 1e-3 → 2e-4)

In Phase 1, only the FC head was trained with Adam at `lr=1e-3` — the frozen backbone provided stable feature extraction. In Phase 2, `layer4` was unfrozen with a reduced `lr=2e-4`, allowing fine-grained adaptation. Validation AUC climbed steadily to **0.9507**. A cosine annealing scheduler and gradient clipping were applied throughout.

Experiment 2 — SGD with Momentum (lr: 1e-2 → 5e-4)

SGD with momentum 0.9 used the same 2-phase structure. SGD requires a higher initial LR (`1e-2` vs `1e-3` for Adam) due to the absence of adaptive per-parameter scaling. Convergence was slower and the model reached only ∼0.79 AUC within the 10-epoch budget, indicating SGD needs more epochs to match Adam under this fine-tuning setup.

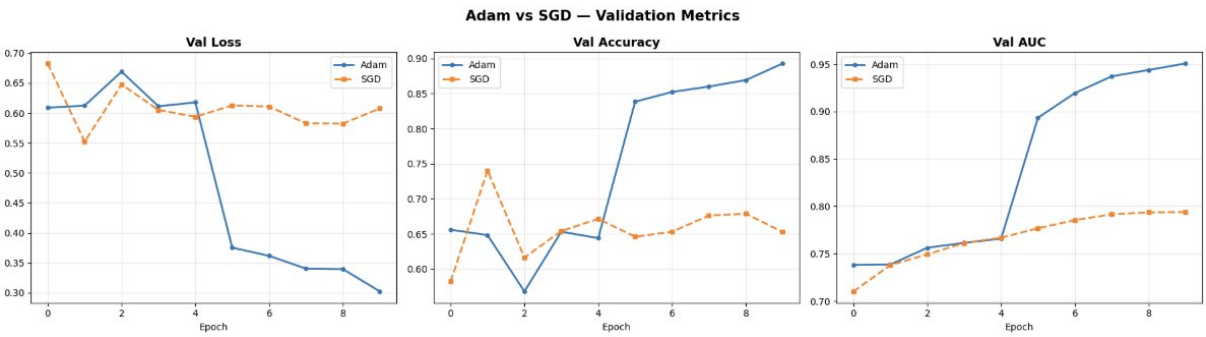


Figure 5: ResNet-152 — Adam vs SGD: Validation Loss, Accuracy, and AUC

Test Set Evaluation — ResNet-152

The Adam model (best val AUC-ROC = 0.9507) was evaluated on the held-out test set. Since the dataset is imbalanced (real:fake ≈ 1:3), **ROC-AUC is used as the primary evaluation metric** as it measures discrimination ability independent of class distribution.

Table 6: ResNet-152 Test Set Metrics (primary metric: ROC-AUC)

Metric	Score	Note
ROC-AUC	**0.9436**	**Primary metric — imbalance-robust**
F1-Score	0.9145	Harmonic mean of precision & recall
Precision	0.9485	
Recall	0.8829	
Accuracy	0.8731	Can be misleading under imbalance

Table 7: ResNet-152 Classification Report

Class	Precision	Recall	F1-Score	Support
Real	0.68	0.84	0.75	2621
Fake	0.95	0.88	0.91	8720
Overall Accuracy	**87.31%**			

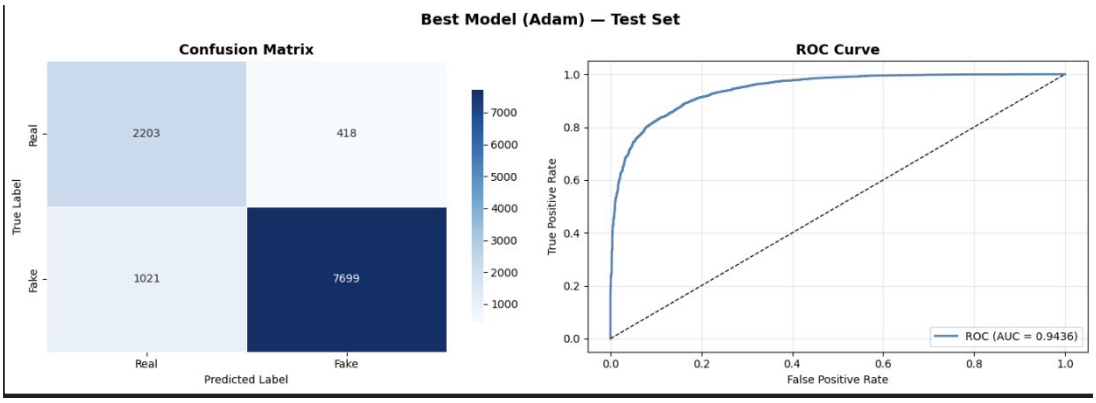


Figure 6: ResNet-152 (Adam) — Confusion Matrix and ROC Curve (AUC = 0.9436)

Observations — ResNet-152

- **Adam outperformed SGD** significantly under the same 10-epoch budget, due to its adaptive per-parameter learning rates.
- **Phase 1 (FC only)** warmed up the classification head quickly — even with the full backbone frozen, ImageNet features provided a strong starting point.
- **Phase 2 (FC + layer4)** delivered the biggest accuracy jump. Unfreezing `layer4` allowed the model to adapt high-level spatial features to deepfake-specific artifacts.
- **Precision for Fake is very high (0.95)**, meaning almost all predicted fakes are truly fake.
- The **Real class recall (0.84)** is far better than ResNet-34's 0.21, demonstrating the strength of pretrained features even with minimal fine-tuning.
- Class imbalance (8720 fake vs 2621 real) caused some bias; class-weighted loss partially mitigated this.
- SGD's lower AUC ($\sim$0.79) is not a fundamental weakness — it simply requires more epochs and careful LR warm-up to match Adam.

Model Comparison

Table 8: ResNet-34 vs ResNet-152 — Final Test Metrics (ranked by primary metric: ROC-AUC)

Metric	ResNet-34 (Scratch)	ResNet-152 (Pretrained)
ROC-AUC (primary)	0.8040	**0.9436**
F1-Score	0.8802	**0.9145**
Accuracy	0.7962	**0.8731**
Precision (Fake)	0.80	**0.95**
Recall (Fake)	**0.97**	0.88
Training Epochs	5	10

Since the dataset is imbalanced (real:fake $\approx$ 1:3), **ROC-AUC is the primary comparison metric**. ResNet-152 outperforms ResNet-34 by **+0.14 AUC points** — a substantial gain driven by ImageNet pretrained features. ResNet-34 has higher Fake recall (0.97 vs 0.88) but at the cost of many false positives on the Real class, which ROC-AUC correctly penalises.

Summary of Learnings

1. **Transfer learning is highly effective.** ResNet-152 pretrained on ImageNet achieves 0.94 AUC in just 10 epochs, while ResNet-34 trained from scratch reaches only 0.80 AUC in 5 epochs.

2. **Progressive fine-tuning reduces training time without sacrificing performance.** By freezing most of the backbone and only training the head + last 1–2 blocks, training time was cut by roughly 10× compared to full fine-tuning.
3. **Adam converges faster than SGD** in fine-tuning settings with a limited epoch budget. SGD is competitive but requires more careful LR selection and more epochs.
4. **Learning rate is the most important hyperparameter.** In ResNet-34 experiments, dropping LR from `1e-3` to `1e-4` improved validation accuracy by ∼0.8%, while a larger batch size further stabilized training.
5. **Class imbalance (real:fake ≈ 1:3) must be addressed.** Class-weighted loss helped, but the Real class still had lower recall, especially in ResNet-34. Oversampling or focal loss could further improve this.
6. **AUC is a more reliable metric than accuracy** when classes are imbalanced. A model biased toward predicting Fake would score high accuracy but low AUC.
7. `layer4` **is the most valuable block to unfreeze** in ResNet-152 — it contains high-level spatial features most relevant to detecting facial manipulation artifacts.

Conclusion

This assignment demonstrated that deep residual networks are effective for deepfake detection. The **ResNet-152 pretrained model with Adam optimizer** achieved the best results, evaluated primarily on **ROC-AUC** due to class imbalance:

- **0.9436 ROC-AUC** *(primary metric)*
- **0.9145 F1-score**
- **87.31% test accuracy**

The **ResNet-34 manual implementation** demonstrated the core mechanics of residual networks and achieved solid performance (79.6% accuracy, 0.80 AUC) purely from scratch, validating the architectural design.

Together, the two models highlight the trade-off between *training from scratch* (full control, interpretable, slower) and *transfer learning* (faster, higher accuracy, dependent on pretrained features).